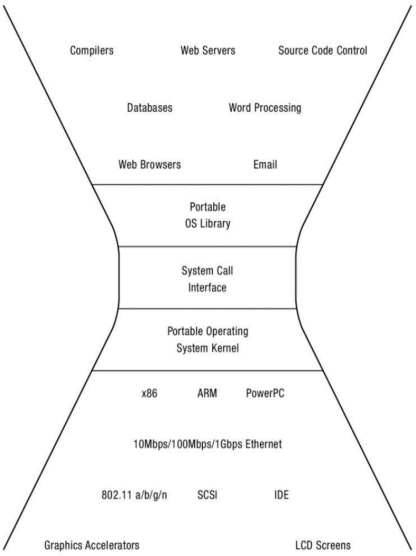
Processes

Description

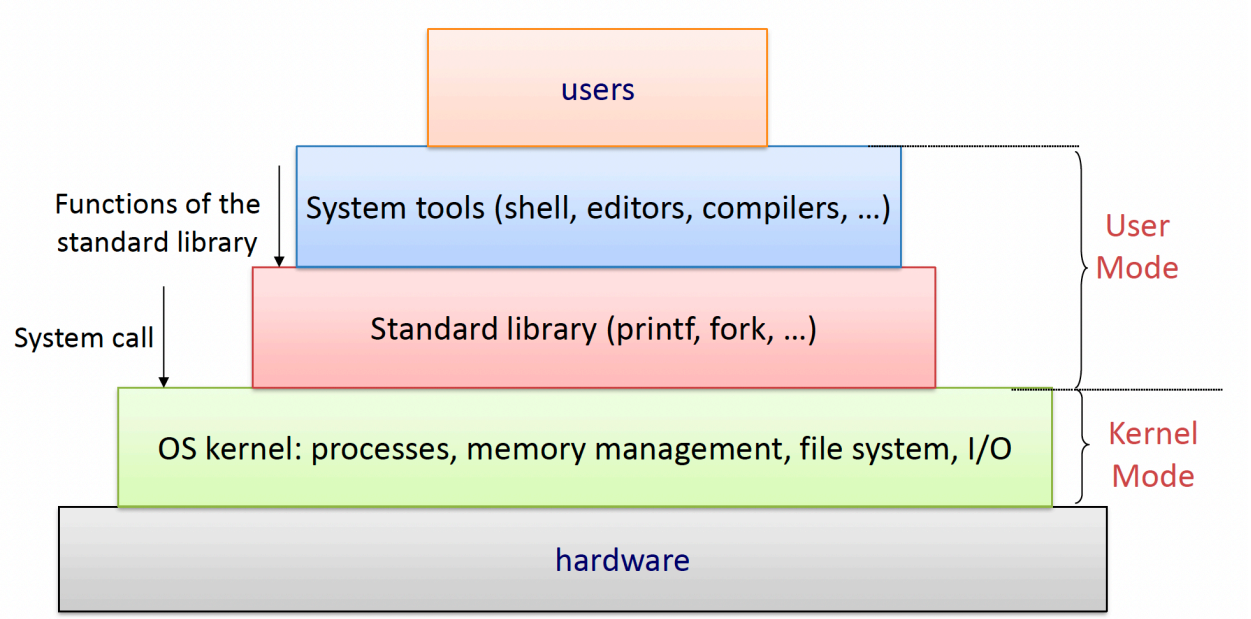
OS Interface Structure



- The OS provides a set of ***system calls*** **(SC)** to applications.
- Internal evolution of the OS is possible without breaking existing software.
- "Thin waist" architecture:
  - **Above**: User applications and libraries.
  - **Below**: Kernel and HAL (Hardware Abstraction Layer).

UNIX Architecture

- **Kernel**: process management, memory, file system, I/O.
- **Standard Libraries**: printf, fork, exec.
- **System Tools**: shell, editors, compilers.
- **System calls** mediate between the kernel and user space.



Process Creation and Management

Main System Calls

- **Process Creation**: `fork()`, `exec()`, `wait()`, `exit()`.
- **I/O**: `open()`, `read()`, `write()`, `close()`.
- **Interprocess Communication**:
  - **Message Passing**: `pipe()`, `dup()`, `socket()`, `select()`, `connect()`.
  - **Shared Memory**: `sem_init()`, `shm_open()`, `mmap()`, `unlink()`.

Shell

- Job control system.
- Allows execution and management of programs.
- Example of compiling a C program:

  Each command creates a new process.

```
$ cc -c file1.c
$ cc -c file2.c
```

```
$ ln -o prog file1.o file2.o
```

Process Creation in Windows

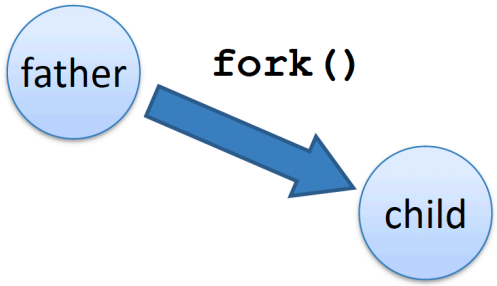
- `CreateProcess()` system call:
  - Initializes the **Process Control Block (PCB)**.
  - Creates and initializes the **address space**.
  - Loads the program and arguments into memory.
  - Informs the **scheduler** that the process is ready.

Process Creation in UNIX

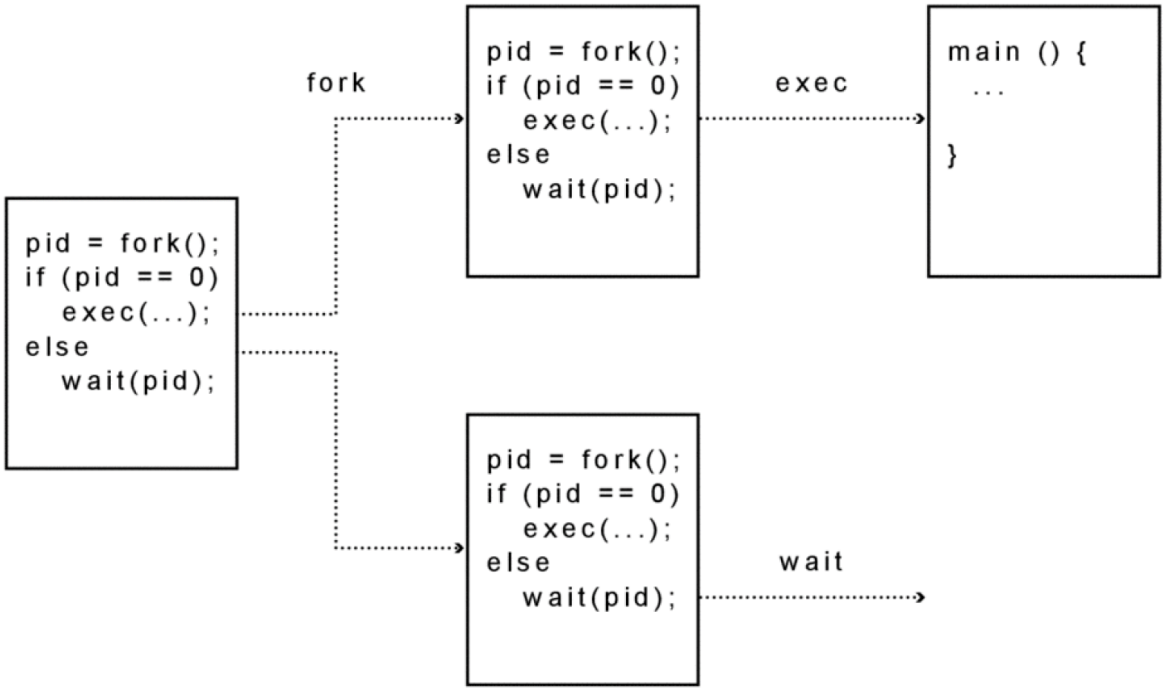
- `fork()`: duplicates the calling process.
- `exec()`: replaces the process code.
- `wait()`: waits for a child process to terminate.
- `signal()`: manages process signaling.

`fork()` Functionality

- Takes no parameters.
- Returns an integer (`pid_t`):
  - **0** in the child.
  - **Child's PID** in the parent.
  - **Negative value** on error.
- Generated processes share the code.
- Distinct address spaces.

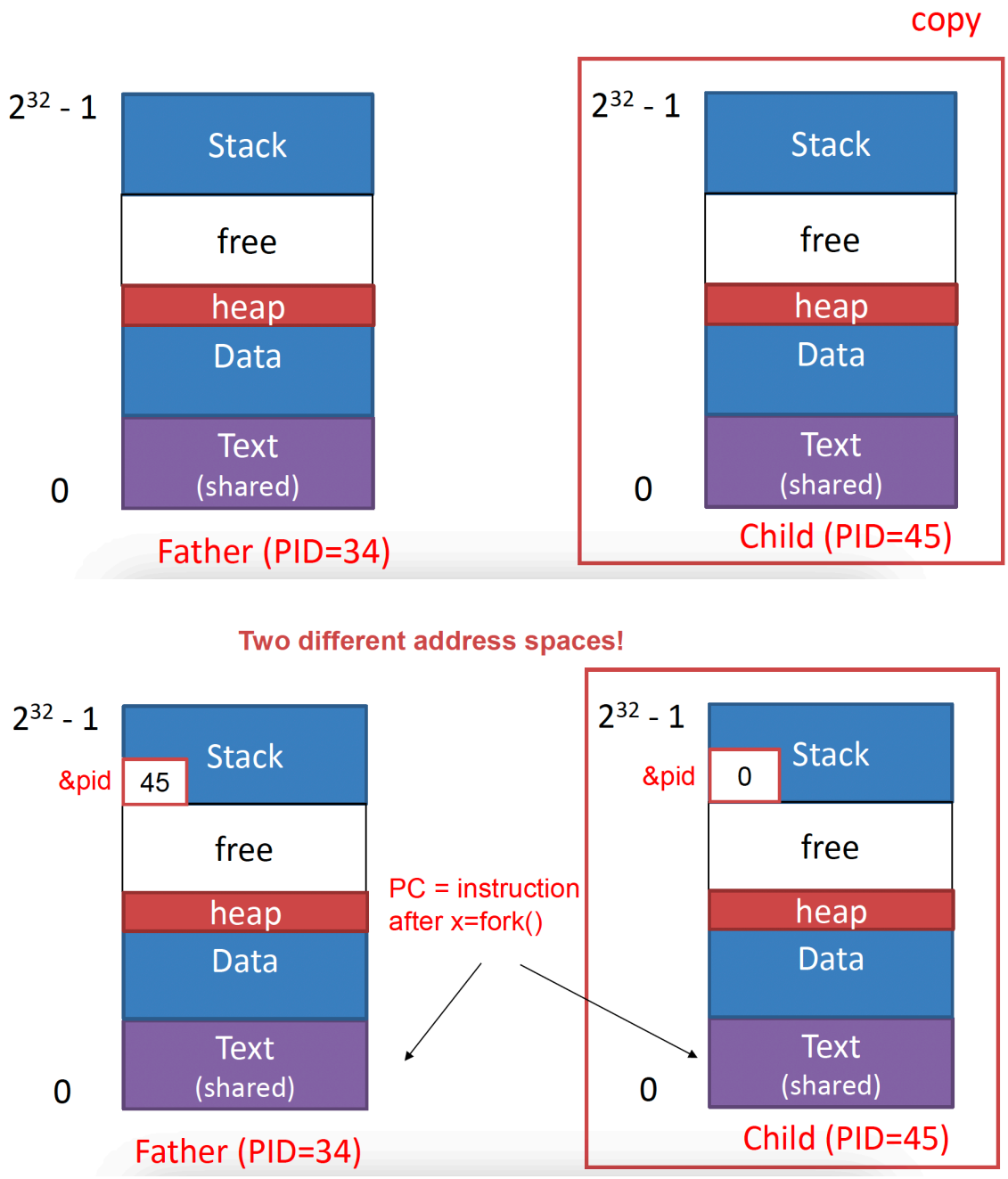


UNIX Process Management



`fork()` Implementation

1. Create the PCB.
2. Create a new address space.
3. Copy the parent's memory.
4. Inherit execution context (open files, environment variables).
5. Scheduler marks the process as "ready".



`exec()` Functionality

- Replaces the executed code of a process.
- Keeps the same **PID** and **PCB**.
- Resets pending signals.

- Keeps open files and kernel stack.
- Returns an error if it fails.

Process Termination

- **Causes**:
  - Exceptions due to illegal actions.
  - Call to `exit()`.
  - Forced termination by parent (`kill`).
- Parent can retrieve the exit code with `wait()`.
- If the parent does not call `wait()`, the child enters **zombie** state.

Simplified Shell Example

```
char *prog, **args;
int child_pid;
while (readAndParseCmdLine(&prog, &args)) {
  child_pid = fork();
  if (child_pid == 0) {
    exec(prog, args); // Child executes the program
  } else {
    wait(child_pid);  // Parent waits
  }
}
```

UNIX Input/Output

File System Interface

- **Uniformity**: same system calls for files and devices.
- **Explicit opening required** with `open()`.

- **Core operations**:
  - `open()`
  - `close()`
  - `read()`
  - `write()`
- **Explicit closure** to release resources.

`open()`

- Returns a **file descriptor**.
- Opening modes:
  - If the file does not exist, error or creation.
  - If it exists, normal opening or content deletion.